

P2406R1

Fix ``counted_iterator`` interaction with input iterators

Draft Proposal, 2021-07-25

This version:

<https://wg21.link/P2406>

Authors:

[Yehezkel Bernat](#)

[Yehuda Bernat](#)

Audience:

SG9 (RANGES)

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Source:

[GitHub](#)

Abstract

``counted_iterator`` increments its internal iterator even when reaching its own end, which makes it unusable in some cases, especially for input iterators. This paper suggests new tools to improve the situation

Table of Contents

- 1 **Revision History**
- 2 **Intro**
- 3 **Problem description**
 - 3.1 Range with the exact number of items

3.2 `input_iterator` case

4 Current behavior is what the standard mandates

5 Desired behavior

6 Naive design of the solution

7 Designs that were considered (and rejected)

7.1 Don't increment the underlying iterator until really required

7.2 Fix `counted_iterator` behavior around 0

8 P2578 and this paper

9 Design of the proposed solution

9.1 The main changes

9.2 Why `lazy_take` instead of fixing `take`?

9.3 `random_access_iterator` case kept as-is

9.4 Discussing `base()` changes

9.5 Constructor with count

10 Proposed Wording

11 TODO

12 Note about optimization

13 Acknowledgements

References

Informative References

§ 1. Revision History

r1: Improving many parts, following feedback from Inbal Levi and from Reddit users

r0: initial revision

§ 2. Intro

Project Euler is a project with many mathematical-related questions that are intended to encourage the reader to write a small program to compute the result. In this case, one of the problems there, no. 37 [EULER], helped reveal a pitfall coming from the definition of `std::counted_iterator`.

§ 3. Problem description

§ 3.1. Range with the exact number of items

Look at this example code [CE-FILTER]:

```
#include <ranges>
#include <iostream>

namespace rv = std::views;

int main() {
    for (auto i : rv::iota(0)
        | rv::filter([](auto i) { return i < 11; })
        | rv::take(11))
        std::cout << i << '\n';
}
```

Compiler explorer gets a timeout when trying to run this simple example, instead of printing the numbers from 0 to 10. Running the same code locally, it runs for very long time. Tracking the roots of the issue, the problem is that `take` uses `counted_iterator` when the range isn't `random_access` and `counted_iterator` increments the internal iterator even if the counter has reached the requested count. In this case, the filter never returns when trying to increment it once again (at least not until `iota` reaches the UB case of signed overflow).

The example above is just for illustration, but we can think about cases where it isn't clear for the user how many items the filter is expected to return, so limiting the output count with `take` becomes dangerous and results in unexpected behavior.

The problem mentioned in the intro is one that actually describes a filter that return exactly 11 elements, so trying to use `take(11)` on it means the program never ends even while it got 11 elements already.

It means `take` isn't usable on ranges that have the exact count of items (and so is `counted_iterator` when wrapping around an iterator for such a range).

§ 3.2. `input_iterator` case

Even more common problem is when using input ranges, e.g. `basic_istream_view`. In these cases, advancing the internal iterator when reaching the count means eating an additional input that can't be retrieved again later, or hanging forever if no additional input exists and the stream isn't closed. For example [\[CE-ISTREAM\]](#):

```
#include <ranges>
#include <iostream>
#include <sstream>
#include <cassert>

namespace rn = std::ranges;
namespace rv = rn::views;

int main()
{
    auto iss = std::istringstream("0 1 2");
    for (auto i : rn::istream_view<int>(iss)
        | rv::take(1))
        std::cout << i << '\n';
    auto i = 0;
    iss >> i;
    std::cout << i << std::endl; // flush it in case the assert fails
    assert(i == 1); // FAILS, i == 2
}
```

It means that one can't use ranges when parsing input, for example.

Seems like this was discussed in [\[range-v3-issue57\]](#), and there was no decision what is the right solution.

§ 4. Current behavior is what the standard mandates

Under 23.5.6.5 [\[counted.iter.nav\]](#), the standard defines the behavior of `operator++` for `counted_iterator` as:

Effects: Equivalent to:

```
++current;  
--length;  
return *this;
```

It means that even when `length` becomes 0, the internal iterator is incremented, thus consuming an additional item from the range, and causing the effects mentioned above for input iterator case or when `++` on the internal iterator is costly (or never returns).

§ 5. Desired behavior

As long as `counted_iterator` is valid (not equal to `default_sentinel`), it must never try to access more than `n` items (when `n` is the given count). If the range doesn't have `n` items, the behavior is kept as is, i.e. it isn't defined (`operator++` might hang forever or access things that shouldn't be accessed etc.).

§ 6. Naive design of the solution

Basically, what is required to implement the desired behavior is changing the behavior of `counted_iterator` operators around 0 length so it doesn't increment the internal iterator when reaching 0 length.

§ 7. Designs that were considered (and rejected)

§ 7.1. Don't increment the underlying iterator until really required

We could change `counted_iterator` internal behavior to increment the underlying iterator only on first dereference. This can't work as copying `counted_iterator` of non-forward iterator (that wasn't dereferenced yet for the current item) and then dereferencing one of them resulted with invalidation of the other one. Even if we are willing to make `counted_iterator` non-copyable, making the dereference operator non-const only or making the internal iterator `mutable` brings with it various other usability and correctness issues.

§ 7.2. Fix `counted_iterator` behavior around 0

Instead of introducing new type, `lazy_counted_iterator`, which is almost like `counted_iterator` but with some changes, we tried to apply similar changes to `counted_iterator` itself. This has been proven to be impossible, even if the changes can be done in ABI compatible way. For example, there is no sane way to fix the c-tor that takes iterator and count, when the count is 0. Here are the details:

`counted_iterator` allows constructing with 0 as argument for length. If we change `counted_iterator` operators to behave similarly to what we propose for `lazy_counted_iterator`, this constructor puts the iterator in an inconsistent internal state, as the underlying iterator is expected to be "one step back".

Please note that `base()` and `--` are the only operations involving the state of the internal iterator and still legal for `counted_iterator` constructed with `n==0`.

The options we considered and rejected are:

Option 1: Require that if `n == 0`, `i` must be decrementable, and actually decrement it in the c-tor. Please note that this obviously works only for `bidirectional_iterator`. Other kind of iterators can be left as UB, or just advice against calling `base()` on the resulted `counted_iterator` (which doesn't sound correct, blocking a basic operation like `base()`).

This option assumes the only reason to create such an `counted_iterator` is to decrement it later, so we always expect the given iterator to be decrementable. Obviously, we can't really assume it. The next operation could be to test for the value of `n` before doing anything else and do nothing in the 0 case.

Option 2: Require that if `n==0`, `i` must be "the one before" the actual iterator (leaving it to the user to decide how to handle, and if neither `--` nor `base()` are ever called on it, it doesn't matter what the user does). This option changes behavior of existing code so it's isn't relevant either.

Option 3: Mark this case internally (e.g. with `length=-1` or a boolean flag) and handle specially when decrementing (`length` "jumps" to 1 after decrementing the internal iterator). Please note that both the counter/flag and the internal iterator must be mutable (to be changed on first access to `base()`). Also, if -1 is used, instead of a separated flag, comparison operators have to consider this case too. Using a flag, OTOH, will probably push for separated specialization of the whole class, so for random-access iterators this member will not exist. This still doesn't solve the problem of invalidation of one copy when `base()` is called on the other one. It also breaks ABI.

Another problem is that keeping the current behavior of `base()` requires `base()` to be non-const (obviously undesired for read-only method) or to start returning by-value, preventing the usage of

`counted_iterator` on move-only iterators [LWG3391].

Our conclusion is that `counted_iterator` can't be fixed to handle all cases.

§ 8. P2578 and this paper

We propose a fix constructed from 2 parts:

1. Block the obviously wrong usages from `counted_iterator` (most usages with input iterators)
2. Create new tools that behave better in these cases

To make it easier to discuss and adopt each part by itself, if needed, we separated them to 2 papers. [P2578] introduces `lazy_weakly_incrementable` concept and blocks usage of `counted_iterator` with input (non-forward) iterators that aren't `lazy_weakly_incrementable`. This paper is about the second part of the solution and builds on the concept introduced with P2578.

§ 9. Design of the proposed solution

§ 9.1. The main changes

We propose adding a new iterator type, `lazy_counted_iterator`. This type behaves similarly to `counted_iterator`, with changes to its operator definition around 0 length so it doesn't increment the internal iterator when reaching 0 length, and as a result doesn't decrement it when getting back from 0 to 1 length. This requires changing `base()` behavior too.

Additionally, this requires adding `lazy_take` and `views::lazy_counted` that uses the new iterator instead of `counted_iterator`.

§ 9.2. Why `lazy_take` instead of fixing `take`?

We could have change `take` to use `lazy_counted_iterator` when constructed with input (non lazy) range. Besides ABI considerations, we find it wrong if `take` used to return one type (`counted_iterator`) and now will start returning a different one, `lazy_counted_iterator`, as this is source-breaking change. Additionally, as demonstrated above, there are cases where the user wants using `lazy_counted_iterator` on forward iterators too, but this is something that only the user know and we can't automatically detect and decide on behalf of them. We can't change all cases

of `take` to use `lazy_counted_iterator`, due to the differences in behavior both for lazy input iterators and forward iterators (that are not random access), as described below.

We aren't happy with the additional burden on teachability, but we believe in most cases users can just use `lazy_take` and it does The Right Thing. The only point where users must be aware of it is when they use `base()` method, which we expect to be quite advance usage in general.

§ 9.3. `random_access_iterator` case kept as-is

To reduce the amount of changes required, we keep the current behavior for `random_access_iterator` case, so we don't have to touch the additional operators defined only for this category. The rational behind it is that for `random_access_iterator` case we can expect the view to either have all the items ready or able to compute all of them efficiently, so it doesn't suffer from an issue similar to the one `forward_iterator` might have.

§ 9.4. Discussing `base()` changes

To keep `lazy_counted_iterator::base()` as `const` member, its behavior is defined such when `count() == 0` it returns the "one before" iterator. If we want it to return the actual end, it means we must advance the underlying iterator first. This doesn't allow `base()` to be `const`, invalidates other copies of the iterator and (depending on the way it's defined) might even make calling `base()` twice UB. This is why we propose the behavior just described.

As mentioned, for `random_access_iterator` we do increment the underlying iterator when `count()` becomes 0 to simplify the definition of other operators (e.g. `+=`). To keep `base()` functionality consistent for all `lazy_counted_iterators`, we return the `prev` of the underlying iterator in this case. As a result, `lazy_counted_iterator::base()` returns by value. As mentioned in [LWG3391], this prevents using it with move-only iterators. Please note that it's OK to call `prev` on the underlying iterator, as `lazy_counted_iterator` can't be created with `count() == 0`, so we are sure there is a `prev`.

To rectify this, tools built on `lazy_counted_iterator` (e.g. `lazy_take`) must access the underlying iterator directly, instead of using `base()`. Users who need to access `base()`, can choose between using `take` with their move-only iterators (if the behavior is right, i.e. they know there is an additional available element at the end, they are willing to throw away the rest of the range anyway and the iterator is lazy) or creating another iterator from the same source they gave to `lazy_take` (as the internal state has changed to point to the last item used by `lazy_take`). Please note that creating such an iterator in the middle is impossible (it would invalidate the underlying iterator of

`lazy_take`), but as only the current element is available in such ranges, it's available already from what `lazy_take` returns in the current iteration.

§ 9.5. Constructor with count

Following the discussion above about the c-tor that takes count, we disallow creating `lazy_counted_iterator` with `n == 0`. To make it easier to use it with pipelines, we might want to explicitly allow it, but disallow any operation on such an iterator that access the underlying iterator or comparing to another iterator (effectively allowing only `count()` and comparing to sentinel).

§ 10. Proposed Wording

Duplicate "Counted iterators" section (25.5.6 [`iterators.counted`]), renaming it to "Lazy counted iterators" (adding 25.5.x [`iterators.lazy.counted`]), with the following differences (the wording here is done against the suggested changes in P2578R0):

Every occurrence of `counted_iterator` becomes `lazy_counted_iterator`

Every stable reference is prefixed with "lazy."

Under 25.5.x.1 [`lazy.counted.iterator`]:

```
template<input_or_output_iterator I>
requires forward_iterator<I> || lazy_weakly_incrementable<I> ||
!input_iterator<I>
class lazy_counted_iterator
```

Under 25.5.x.2 [`lazy.counted.iter.const`]:

```
constexpr counted_iterator(I i, iter_difference_t<I> n);
```

Preconditions: ~~`n >= 0`~~ `n > 0`.

Under 25.5.x.3 [`lazy.counted.iter.access`]:

```
constexpr const I & base() const &;
```

Effects: Equivalent to: `return current;`

```
constexpr I base() const &;
```

```
requires random_access_iterator<I>;
```

Effects: Equivalent to: `return length ? current : std::ranges::prev(current);`

```
constexpr I base() &&;
```

Returns: `std::move(current)`.

```
constexpr I base() &&;
```

```
requires random_access_iterator<I>;
```

Returns: `length ? std::move(current) : std::ranges::prev(current)`.

Under 23.5.6.5 [**counted.iter.nav**]:

```
constexpr lazy_counted_iterator& operator++();
```

Preconditions: `length > 0`.

Effects: Equivalent to:

```
if (length > 1) ++current;
```

```
--length;
```

```
return *this;
```

```
constexpr lazy_counted_iterator& operator++();
```

```
requires random_access_iterator<I>;
```

Preconditions: `length > 0`.

Effects: Equivalent to:

```
++current;
```

```
--length;
```

```
return *this;
```

```
decltype(auto) operator++(int);
```

Preconditions: `length > 0`.

Effects: Equivalent to:

```
--length;
```

```
try { return current++; }
```

```
try { return length ? current++ : current; }
```

```
catch(...) { ++length; throw; }
```

```
constexpr lazy_counted_iterator operator++(int)
```

```
requires forward_iterator<I>;
```

Effects: Equivalent to:

```
lazy_counted_iterator tmp = *this;
```

```
++*this;
```

```
return tmp;
```

```
constexpr lazy_counted_iterator& operator--()
```

```
requires bidirectional_iterator<I>;
```

Effects: Equivalent to:

```
if (length) --current;  
++length;  
return *this;
```

```
constexpr lazy_counted_iterator& operator--()  
requires random_access_iterator<I>;
```

Effects: Equivalent to:

```
--current;  
++length;  
return *this;
```

§ 11. TODO

Missing wording for `lazy_take` and `views::lazy_counted` and adding more descriptions to `lazy_counted_iterator` (e.g. behavior of `base()`).

§ 12. Note about optimization

It's interesting to note that with any level of optimization enabled (including `-Og`!), gcc is able to "fix the issue" [\[CE-OPT\]](#) for the filter+take case (but not for `input_iterator`, of course). It's maybe even more interesting to see the mentioned optimization is not an optimizer bug, and when the filter will never return another number, it doesn't change the behavior [\[CE-OPT2\]](#).

§ 13. Acknowledgements

Many thanks to the Israeli NB members for their feedback and support, in particular Inbal Levi, Dvir Yitzchaki and Dan Raviv. Thanks r/cpp Reddit users for their [feedback](#) on P2406R0.

§ References

§ Informative References

[CE-FILTER]

filter+take problem example, Compiler Explorer. URL: <https://gcc.godbolt.org/z/9TjbdMn3d>

[CE-ISTREAM]

istream problem example, Compiler Explorer. URL: <https://gcc.godbolt.org/z/Eb8rdWYbP>

[CE-OPT]

Optimizer magic solves filter+take issue, Compiler Explorer. URL:
<https://gcc.godbolt.org/z/4dahzG8Gz>

[CE-OPT2]

Optimizer is right when filter really never returns, Compiler Explorer. URL:
<https://gcc.godbolt.org/z/PvMY8WeaT>

[EULER]

Truncatable primes, problem 37, Project Euler. URL: <https://projecteuler.net/problem=37>

[LWG3391]

LWG3391 - Problems with counted_iterator/move_iterator::base() const &. URL:
<https://cplusplus.github.io/LWG/issue3391>

[P2578]

Block eager input (non-forward) iterators from counted_iterator. URL: <https://wg21.link/p2578>

[RANGE-V3-ISSUE57]

range-v3 - istream_range filtered with take(N) should stop reading at N. URL:
<https://github.com/ericniebler/range-v3/issues/57>

[REDDIT-CPP]

r/cpp comments on P2406R0. URL:
https://www.reddit.com/r/cpp/comments/orw4q8/wg21_july_2021_mailing/h6kqu7y